

Revisão — Álgebra Booleana e Mapa de Karnaugh

Disciplina: Sistemas Digitais **Material de origem:** slides Álgebra Booleana (79 lâminas) e Mapa de Karnaugh (78 lâminas) **O que este documento é:** a matéria dos dois conjuntos de slides reorganizada em ordem de estudo, com **todos os exercícios propostos resolvidos**, os erros dos slides conferidos, e um simulado com gabarito no fim.

Convenção de notação usada aqui. Os slides usam a barra sobre a variável para complemento. Em texto corrido a barra não sobrevive, então uso o **apóstrofo**: A' lê-se “A barrado” / “não A”. Quando a barra cobre mais de uma variável, uso parênteses: $(AB)'$ é a barra longa sobre AB — que **não** é a mesma coisa que $A'B'$. Essa distinção é o centro do teorema de DeMorgan, então vale prestar atenção nela desde já.

Índice

#	Parte	Assunto
1–2	I	Por que $1 + 1 = 1$; booleano não é binário
3	I	Aritmética booleana: OR é soma, AND é produto
4–5	I	As 8 identidades, complemento duplo e as 3 propriedades
6	I	As regras exclusivas da álgebra booleana
7	I	XOR — a porta que não tem equivalente algébrico
8	I	Teoremas de DeMorgan e a arte de quebrar barras
9–10	I	Tabela-verdade $\rightarrow$ expressão: SOP e POS; o incinerador
11–12	II	De Venn ao K-map; código Gray
13–15	II	Agrupamento, bordas e cantos
16–17	II	Maxtermos (POS) e <i>don't care</i>
18	—	Erros e pegadinhas conferidos
19	—	Exercícios dos slides, resolvidos
20	—	Simulado + gabarito

PARTE I — ÁLGEBRA BOOLEANA

1. A ideia central: as “leis” dependem da definição de número

Isto é o fio condutor dos primeiros 14 slides e cai em prova como pergunta conceitual.

A afirmação $3 + 4 = 7$ parece uma verdade absoluta, mas só é verdade **dentro do escopo dos números reais**. Se você anda 3 m para o norte e 4 m para o leste, a distância em linha reta ao ponto inicial é 5 m, não 7 — porque grandezas com direção somam como vetores, não como escalares. As regras da aritmética não foram quebradas; o que mudou foi **o conjunto sobre o qual elas operam**.

O mesmo vale na lógica. Aristóteles fundou um sistema com apenas dois valores de verdade (verdadeiro/falso), e desse recorte bivalente saem as quatro leis clássicas:

1. **Lei da Identidade** — A é A
2. **Lei da Não Contradição** — A não é não- A
3. **Lei do Terceiro Excluído** — ou A , ou não- A
4. **Lei da Inferência Racional**

Essas leis valem *onde uma proposição só pode ter um de dois valores*. Na **lógica fuzzy** (multivalorada), onde uma proposição é verdadeira “em certo grau”, a Lei da Não Contradição deixa de valer — e não há escândalo nenhum nisso, porque o escopo é outro.

George Boole (1815–1864) deu forma simbólica ao sistema de Aristóteles no tratado de 1854 “*Uma Investigação sobre as Leis do Pensamento*”. **Claude Shannon**, do MIT, na tese de 1938 “*Análise Simbólica de Circuitos de Relés e Comutação*”, percebeu que essa álgebra descrevia exatamente circuitos liga-desliga — e foi isso que transformou um trabalho de filosofia matemática na ferramenta de projeto de todo circuito digital que existe.

Conclusão que cai em prova: $1 + 1 = 1$ não contradiz nada. No mundo booleano só existem 1 e 0. A soma $1 + 1$ certamente não é 0, logo — **por eliminação** — é 1.

2. Números booleanos não são números binários

Esta é a confusão mais comum e a mais fácil de cobrar.

	Booleano	Binário
O que é	um sistema matemático distinto dos reais	uma notação alternativa para números reais
Tamanho	sempre 1 bit	vários bits, ponderados por posição
10011	não existe	vale dezenove
Símbolos	1 e 0	1 e 0

Os dois se confundem porque usam os mesmos dois símbolos. Mas o número binário 10011 não tem lugar no mundo booleano, assim como 2 ou 32 não têm.

3. Aritmética booleana

3.1 Adição = porta OR

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 1 \quad \leftarrow \text{a que confunde}$$

Não importa quantos termos: $1 + 1 + 1 + 1 = 1$. É exatamente a tabela-verdade de um OR.

3.2 Multiplicação = porta AND

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

Idêntica à multiplicação dos reais: qualquer coisa vezes 0 é 0; qualquer coisa vezes 1 fica inalterada. É a tabela-verdade de um AND.

3.3 O que não existe

- **Não existe subtração.** Subtrair implica números negativos ($5 - 3 = 5 + (-3)$), e quantidades negativas são proibidas no mundo booleano.

- **Não existe divisão**, porque divisão é subtração composta — do mesmo jeito que multiplicação é adição composta.

3.4 Variáveis e complemento

- Variáveis booleanas são **sempre letras MAIÚSCULAS**, nunca minúsculas (ao contrário da álgebra “normal”).
- Toda variável tem um **complemento**: se $A = 0$, então $A' = 1$.

4. As oito identidades (+ complemento duplo)

Identidade = afirmação verdadeira para **todos** os valores possíveis da variável.

Aditivas (OR)

Identidade	Por quê	Leitura em circuito
$A + 0 = A$	igual à álgebra normal	entrada em 0 não faz nada
$A + 1 = 1$	exclusiva do booleano	o 1 <i>sobrepõe</i> A e trava a saída em 1
$A + A = A$	não existe “2A”	ligar as duas entradas de um OR no mesmo sinal
$A + A' = 1$	um dos dois é 1, e algo + 1 = 1	variável com seu próprio complemento

Multiplicativas (AND)

Identidade	Por quê
$A \times 0 = 0$	igual à álgebra normal
$A \times 1 = A$	igual à álgebra normal
$A \times A = A$	não existe “A ² ” (quadrado implica o número 2)
$A \times A' = 0$	um dos dois é 0, e algo $\times 0 = 0$

Complemento duplo

$A'' = A$ — complementar duas vezes (ou qualquer número **par** de vezes) devolve o valor original. É análogo à negação nos reais: um número par de trocas de sinal se cancela.

5. As três propriedades algébricas

Estas **são iguais** às da álgebra normal — é por isso que a manipulação algébrica funciona.

Propriedade	Adição	Multiplicação
Comutativa	$A + B = B + A$	$AB = BA$
Associativa	$(A+B)+C = A+(B+C)$	$(AB)C = A(BC)$
Distributiva	—	$A(B + C) = AB + AC$

A distributiva lida nos dois sentidos: **expandir** um produto de soma, e — de trás para frente — **fatorar** termos de uma soma-de-produtos. Fatorar é o movimento que mais simplifica na prática.

6. As regras exclusivas da álgebra booleana

Estas **não têm equivalente** nos reais. São as que fazem a simplificação acontecer.

Regra	Nome	Prova
$A + AB = A$	absorção	$A + AB = A(1 + B) = A \times 1 = A$
$A + A'B = A + B$	—	$A + A'B = (A + A')(A + B) = 1 \times (A+B) = A+B$
$(A+B)(A+C) = A + BC$	dual da distributiva (forma POS)	expandir e absorver

6.1 A parte difícil: reconhecer a forma padrão

O obstáculo real não é decorar as regras — é **enxergá-las em expressões que não estão na forma padrão**. Exemplo dos slides: $ABC + 1$ também se reduz a 1, pela identidade $A + 1 = 1$, porque o “A” da forma padrão pode representar o termo inteiro ABC .

6.2 Às vezes é preciso “dessimplificar” primeiro

Um movimento contraintuitivo e cobrado: usar $A + AB = A$ **ao contrário**, transformando um A solto em $A + AB$ para criar um termo que depois se cancela com outro.

Parece um passo para trás, mas é o mesmo tipo de sacrifício calculado de uma partida de xadrez: saber **quando** dar esse passo é a parte artística da álgebra.

7. XOR — a porta sem equivalente algébrico

Falta um elemento no conjunto de operações booleanas: o **OU-exclusivo**.

- OR = adição booleana
- AND = multiplicação booleana
- NOT = complementação
- **XOR = nada**. Não há operação booleana direta equivalente.

Existe símbolo de porta, mas ele quase nunca aparece em expressões booleanas, justamente porque as identidades, leis e regras de simplificação **não se aplicam a ele**.

A saída é reescrever XOR em termos de OR e AND:

$$A (+) B = A'B + AB'$$

Uso prático (isto cai em prova): qualquer expressão na forma $A'B + AB'$ — duas portas AND alimentando um OR — pode ser substituída por **uma única porta XOR**. Isso é redução de 3 portas para 1.

8. Teoremas de DeMorgan

8.1 O enunciado

DeMorgan trata de **complementação de grupo**: o complemento de um *conjunto* de termos, representado por uma barra longa sobre mais de uma variável.

$$\begin{array}{ll} (AB)' = A' + B' & \text{NAND} = \text{Negative-OR} \\ (A+B)' = A'B' & \text{NOR} = \text{Negative-AND} \end{array}$$

Em linguagem de portas: **inverter a saída de uma porta dá a mesma função que a porta do tipo oposto (AND OR) com as entradas invertidas.**

Tabela-verdade que prova o primeiro caso (está no slide 46):

A	B	$(AB)'$	$A' + B'$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

8.2 A barra é um símbolo de AGRUPAMENTO

$(AB)'$ **não é igual a** $A'B'$.

A barra longa, sozinha, já age como parêntese quando cobre mais de uma variável. Como o apóstrofo não se estende sobre duas variáveis, usamos parênteses para marcar que ele se aplica ao termo inteiro. Isso muda profundamente como a expressão é avaliada.

8.3 O procedimento: quebrar a barra

Pense o teorema como **quebrar uma barra longa**:

Quando uma barra longa é quebrada, **a operação diretamente abaixo da quebra** muda de adição para multiplicação (ou vice-versa), e os pedaços da barra permanecem sobre as variáveis individuais.

As três regras de ouro:

1. Quando há várias “camadas” de barras, **quebre uma barra por vez.**
2. Geralmente é mais fácil começar pela **barra mais longa (mais acima).**
3. **NUNCA quebre duas barras num único passo.** Quebrar *uma* barra em **mais de um lugar** ao mesmo tempo é legítimo; quebrar *duas barras diferentes* no mesmo passo, não. Confundir as duas coisas é o erro mais frequente e leva a resultado errado.

Exemplo trabalhado (slide 50): reduzir $(A + (BC)')'$.

$(A + (BC)')'$	barra mais longa cobre tudo
$= A' \cdot ((BC)')'$	quebrou a de cima: + virou ·
$= A' \cdot BC$	complemento duplo: $((BC)')' = BC$
$= A'BC$	

Circuito final: **uma porta AND de três entradas com a entrada A invertida.**

8.4 Fecho de DeMorgan

- NAND Negative-OR; NOR Negative-AND.
- Ao quebrar, a operação sob a quebra se inverte; os pedaços da barra ficam sobre os termos.
- Comece pela barra mais externa. **Jamais duas barras num passo.**
- Barras são agrupamento: ao quebrar, os termos abaixo **continuam agrupados** — use parênteses para não mudar a precedência.

9. Da tabela-verdade para a expressão: SOP e POS

O projeto de um circuito digital quase sempre começa com uma **tabela-verdade** descrevendo o que o circuito deve fazer. Existem dois procedimentos mecânicos para sair dela.

9.1 Soma de Produtos (SOP — *Sum-Of-Products*)

Um **somatório de termos**, cada termo sendo um **produto** de variáveis. Exemplo: $ABC + BC + DF$.

Procedimento:

1. Olhe as linhas da tabela cuja saída é **1**.
2. Para cada uma, escreva o **termo produto** que vale 1 exatamente naquela combinação. (variável = 1 entra direto; variável = 0 entra complementada)
3. **Some** todos os termos produto.

Exemplo: na linha $A=0$, $B=1$, $C=1$, o termo é $A'BC$.

Implementação: um conjunto de portas **AND** (os produtos) alimentando **uma única porta OR** (a soma).

9.2 Produto de Somas (POS — *Product-Of-Sums*)

Um **produto de termos soma**. Exemplo: $(A+B)(C+D)$.

Procedimento:

1. Olhe as linhas cuja saída é **0**.
2. Para cada uma, escreva o **termo soma** que vale 0 exatamente naquela combinação. (variável = 0 entra direto; variável = 1 entra complementada — é o **inverso** do SOP)
3. **Multiplique** todos os termos soma.

Exemplo: na linha $A=0$, $B=0$, $C=0$, o termo é $(A+B+C)$.

Implementação: um conjunto de portas **OR** (as somas) alimentando **uma única porta AND** (o produto).

9.3 Qual escolher

Aquele que tiver menos linhas. Se a tabela tem muitos 1 e poucos 0, o POS sai com menos termos; se tem poucos 1, o SOP sai menor. Os dois descrevem a mesma função.

10. O exemplo completo: incinerador de resíduos tóxicos

Este exemplo atravessa os dois conjuntos de slides — é o caso que amarra a matéria.

O problema. Um incinerador neutraliza resíduos tóxicos (inclusive lixo médico com vírus e bactérias) pelo calor intenso da chama. Enquanto há chama, é seguro injetar resíduo. Se a chama apagar, o resíduo sai pelo exaustor **sem ser neutralizado** e vira ameaça à saúde de quem estiver perto. É preciso detectar a chama e só liberar a válvula com chama “comprovada”.

Redundância. Pelo risco, usam-se **três sensores**, cada um com contato normalmente aberto (aberto sem chama, fechado com chama).

10.1 Primeira tentativa: lógica “todos os três” (AND)

Abrir a válvula **se e somente se** os três sensores detectarem chama.

Saída = ABC — uma porta AND de 3 entradas.

- **Por que não “qualquer um dos três” (OR)?** Porque anularia o propósito da redundância: se um sensor falhar indicando chama falsamente, o sistema se comporta igual a um sistema de sensor único com o mesmo defeito.

- **Vantagem do AND:** um único sensor com falha “alta” não consegue manter a válvula aberta. Seriam necessárias três falhas iguais — cenário altamente improvável.
- **Desvantagem do AND:** se um sensor falhar indicando **ausência** de chama havendo chama boa, a válvula fecha desnecessariamente — perda de tempo de produção e desperdício de combustível (mantendo fogo que não incinera nada).

10.2 Solução: lógica “dois de três”

Abrir a válvula se **pelo menos dois** dos três sensores indicarem boa chama. Atende às duas necessidades: tolera uma falha “baixa” sem desligar, e mantém a redundância contra falha “alta”.

Tabela-verdade:

A	B	C	Boa chama	Falha de sensor
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	1
1	0	0	0	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	0

SOP direto das quatro linhas com saída 1:

$$\text{Saída} = A'BC + AB'C + ABC' + ABC$$

Simplificação (usando $A + A = A$ para reaproveitar ABC três vezes):

$$\begin{aligned}
 &= (A'BC + ABC) + (AB'C + ABC) + (ABC' + ABC) \\
 &= BC(A' + A) + AC(B' + B) + AB(C' + C) \\
 &= BC + AC + AB
 \end{aligned}$$

Resultado: $\text{Saída} = AB + BC + AC$ — três portas AND de 2 entradas alimentando um OR de 3 entradas, em vez de quatro AND de 3 entradas. (Esta é a clássica **função maioria**.)

10.3 O detector de falha de sensor (POS)

Se o incinerador está desligado (sem chama) e um ou mais sensores ainda indicam chama — combinações 001, 010, 011, 100, 101, 110 — há **problema definitivo de sensor**. O circuito de falha acusa sempre que os sensores **discordam**.

Saída 0 só em duas linhas: 000 (todos concordam: sem chama) e 111 (todos concordam: com chama). Como há poucos zeros, o **POS** é o caminho econômico:

$$\text{Falha} = (A + B + C) \cdot (A' + B' + C')$$

Duas portas OR de 3 entradas alimentando um AND de 2 entradas.

PARTE II — MAPA DE KARNAUGH

11. Por que o K-map existe

nº de variáveis	ferramenta mais rápida
1–2	álgebra booleana
3	ainda usável na álgebra, mas já mais lento
4+	mapa de Karnaugh — a álgebra vira trabalho braçal

Maurice Karnaugh (1924–2022), engenheiro de telecomunicações, criou o mapa nos **Bell Labs em 1953**, projetando circuitos de comutação telefônica com lógica digital.

“Simplificar” aqui tem definição operacional de custo: **o menor número de portas, com o menor número de entradas por porta**. Menos componentes = maior velocidade (menos atraso de propagação), menor consumo, menor custo, maior confiabilidade.

12. A ponte: diagramas de Venn → K-map

Um **conjunto** é uma coleção de objetos dentro de um universo. Adotando OR/AND no lugar de união/interseção (terminologia da eletrônica digital), há quatro relações possíveis entre dois conjuntos:

caso	exemplo	significado
disjuntos	$A=\{1,2,3,4\}, B=\{5,6,7,8\}$	nada em comum
A contido em B	$A=\{1,2\}, B=\{1,...,8\}$	A é subconjunto de B
coincidentes	$A=\{1,2,3,4\}, B=\{1,2,3,4\}$	A e B idênticos
parcialmente sobrepostos	$A=\{1,2,3,4\}, B=\{3,4,5,6\}$	o caso interessante

Traduzindo para booleano:

- toda a área hachurada (qualquer hachura) = $A + B$ (OR)
- só a área com hachura dupla = AB (AND)
- fora da região = complemento

Aviso importante dos slides: *diagramas de Venn não provam nada formalmente*. Para prova formal é preciso álgebra booleana. Venn serve para **verificação e visualização** — foi assim que os slides visualizaram DeMorgan mostrando que $(A + B')'$ e AB' ocupam exatamente a mesma região.

A transformação: o círculo A é expandido até preencher o universo retangular, depois vira um retângulo. O que sobra fora é A' . Fazendo o mesmo com B e sobrepondo, cada célula da grade resultante é uma **interseção única** de variáveis. Em K-map **não se usa sombreamento** — escreve-se 0 ou 1 na célula.

Na forma final, os nomes das variáveis ficam junto à linha diagonal: a variável **acima** da diagonal é atribuída às **colunas**, a de **baixo**, às **linhas**. O 0 substitui a forma complementada, o 1 a forma direta.

O K-map é uma tabela-verdade rearranjada. As saídas da tabela correspondem **um-para-um** às células do mapa. Ambos, mais a lógica de relés, o diagrama de portas e a equação booleana, são **cinco descrições equivalentes** da mesma função — use a mais útil para a tarefa.

Um K-map de 3 variáveis tem $2^3 = 8$ células; cada célula é identificada por um termo produto de 3 variáveis.

13. Código Gray: a razão de o mapa funcionar

A sequência no topo do mapa **não é binária** (00, 01, 10, 11). É **código Gray**:

00 01 11 10

Por quê: o código Gray muda **apenas um bit por vez** ao avançar na sequência. Isso garante que **células adjacentes diferem em exatamente uma variável** — que é precisamente a condição para $X + X' = 1$ eliminar essa variável ao agrupar.

Se você montar o mapa em ordem binária, todo o método desmorona. Não é estética: é a propriedade que faz a simplificação funcionar. Esta é a pegadinha nº 1 do assunto.

Para mapas de 4 variáveis, basta copiar a sequência que está no topo do mapa de 3 variáveis para o **lado esquerdo** do mapa de 4.

14. Procedimento de simplificação (SOP / mintermos)

Mintermo: expressão booleana que resulta em **1 numa única célula** e 0 em todas as outras. É o comportamento do SOP.

Os cinco passos:

1. Escreva a expressão booleana do circuito original.
2. Transfira cada termo produto para o mapa (marque 1 nas células).
3. Forme grupos de células adjacentes.
4. Para cada grupo, escreva o termo produto.
5. Desenhe o diagrama lógico simplificado.

Regras de agrupamento:

regra	detalhe
adjacência	acima/abaixo/ao lado — nunca na diagonal
tamanho	sempre potência de 2 : 1, 2, 4, 8, 16
o que fica	as variáveis constantes dentro do grupo
o que sai	as variáveis que mudam dentro do grupo
reutilizar	células podem pertencer a mais de um grupo — e isso é desejável
meta	menos grupos e grupos maiores = resultado mais simples

Relação direta e útil de lembrar:

tamanho do grupo (4 variáveis)	variáveis no termo
1 célula	4
2 células	3
4 células	2
8 células	1
16 células	nenhuma (saída = 1)

Sobre reutilizar células: os slides trazem um exemplo em que existem duas respostas (“Saída” e “Saída Errada”) e avisam que **ambas são logicamente corretas** — os dois circuitos produzem a mesma saída. A diferença é que a primeira é a **solução de menor custo**. Reaproveitar um 1 já coberto para formar um grupo maior é exatamente o que baixa o custo.

15. As adjacências que não parecem adjacências

Este é o segundo grande tropeço. O mapa é uma superfície **enrolada**, não um retângulo plano.

situação	visualização dos slides	resultado
bordas esquerda/direita	“enrole o mapa como um cilindro ”	grupo de 4 com C' em comum
bordas superior/inferior	enrole no outro eixo $\rightarrow$ 8 células adjacentes	grupo de 8 $\rightarrow$ B'
os 4 cantos	“dobre os cantos como um guardanapo ”	grupo de 4 $\rightarrow$ B'D'

No caso dos quatro cantos: B = 0 nas quatro células e D = 0 nas quatro células; A e C variam, então saem. Resultado: **Saída = B'D'** — um único termo, a partir de quatro mintermos espalhados pelas pontas do mapa.

15.1 Quando uma célula não agrupa com nada

Acontece com frequência em problemas reais. A célula solta **permanece como termo completo** na resposta. Exemplo dos slides: **Saída = B'C'D' + A'B'D' + ABCD** — o ABCD ficou inalterado porque não tinha vizinho.

15.2 Mais de uma solução de custo mínimo

Também é comum. Nos slides há dois casos em que duas coberturas diferentes dão **quatro termos produto de três variáveis cada** — ambas igualmente válidas, custo idêntico. A diferença vem só de como as células foram agrupadas; uma célula solta pode sair como ABC' ou ABD, à sua escolha. **Não existe resposta única**, e uma prova bem feita aceita qualquer uma das mínimas.

16. Maxtermos: a solução POS no mapa

Para toda solução SOP existe também uma **solução POS**, que pode ser mais útil dependendo da aplicação.

Maxtermo: expressão que resulta em **0 numa única célula** e 1 em todas as outras. É um único termo **soma** — por exemplo (A+B+C) aparece como um único 0 num mapa cheio de 1s.

16.1 Colocar um maxtermo no mapa

1. Identificar o termo **soma** a mapear.
2. Escrever o valor numérico binário correspondente.
3. **Formar o complemento.**
4. Usar o **complemento** como endereço para colocar um **0** no mapa.
5. Repetir para os demais maxtermos.

O passo 3 é o que todo mundo esquece. No SOP o endereço é direto; no POS é **complementado**. (A+B+C) vale 0 quando A=0, B=0, C=0 $\rightarrow$ binário 000; mas ao mapear um termo soma você **complementa as variáveis de entrada** para achar a célula certa.

16.2 Extrair a solução POS

1. Formar os **maiores grupos de 0s** possíveis, cobrindo todos os maxtermos. Grupos devem ser potências de 2.
2. Escrever o valor numérico binário do grupo.

3. **Complementar** esse valor.
4. Converter o valor complementado em um **termo soma**.
5. Repetir para os demais grupos. Cada grupo gera um termo soma do resultado POS.

Relação tamanho → termo: grupos maiores geram termos soma com **menos entradas**; menos grupos geram **menos termos soma**. No exemplo dos slides, três grupos → três termos soma: o grupo de 4 células dá um termo de 2 variáveis, e os dois grupos de 2 células dão termos de 3 variáveis.

16.3 SOP x POS lado a lado

	SOP (mintermo)	POS (maxtermo)
olha as linhas com saída	1	0
marca no mapa	1	0
endereço	direto	complementado
agrupa	os 1s	os 0s
cada grupo vira	termo produto	termo soma
resultado	soma de produtos	produto de somas
circuito	ANDs → um OR	ORs → um AND

17. *Don't care* — a ferramenta de otimização

Algumas combinações de entrada **nunca vão ocorrer** ou o resultado delas é **irrelevante** (entradas inválidas). Em vez de atribuir arbitrariamente 0 ou 1 a essas entradas, marca-se **X** (“*don't care*”, tanto faz) na tabela-verdade e no mapa.

O X **não é um marcador passivo** — é elemento ativo de simplificação. Ele dá ao projetista a liberdade de tratá-lo como 0 **ou** como 1 durante a minimização.

O critério é puramente estratégico:

Inclua o X num grupo de 1s **se e somente se** isso permitir formar um grupo **maior** — transformar um par em quarteto, ou um quarteto em octeto.

Se o X não aumenta nenhum grupo, **deixe-o de fora**. Incluir X sem ganho só adiciona cobertura desnecessária.

18. Erros e pegadinhas — conferidos

18.1 Os erros que estão nos próprios slides

Confira estes ao estudar pelo PDF original.

a) Slide 44 (Álgebra Booleana) — **XOR**. O texto diz: “Qualquer expressão que siga a forma $A \sim B + A \sim B$ ”. As duas parcelas aparecem iguais, o que não pode estar certo — se fossem iguais, $A + A = A$ e não haveria XOR nenhum. A forma correta é com os complementos **cruzados**:

$$A (+) B = A'B + AB'$$

b) Slide 35 (Álgebra Booleana) — **trecho em inglês**. “Another rule involves the simplification of a product-of-sums expression” ficou sem tradução. A regra referida é $(A + B)(A + C) = A + BC$.

c) Slide 12 (Álgebra Booleana) — “Existem encontrará muitas semelhanças” — frase truncada na edição. Leia-se “Você encontrará muitas semelhanças”.

18.2 As dez pegadinhas que mais custam nota

#	pegadinha	o correto
1	montar o K-map em ordem binária	código Gray: 00, 01, 11, 10
2	$(AB)' = A'B'$	falso. $(AB)' = A' + B'$
3	quebrar duas barras num passo	uma por vez , da mais externa para dentro
4	agrupar na diagonal	só ortogonal (acima/abaixo/lado)
5	grupo de 3, 5, 6 células	só potências de 2
6	esquecer bordas e cantos	o mapa enrola nos dois eixos
7	não reutilizar células	reutilizar é desejável , baixa o custo
8	no POS, usar endereço direto	no POS o endereço é complementado
9	confundir booleano com binário	1 bit vs. notação posicional
10	achar que existe uma única resposta certa	há problemas com várias soluções de custo mínimo

18.3 Fórmulas em uma tela

IDENTIDADES

$$\begin{array}{ll} A + 0 = A & A \times 0 = 0 \\ A + 1 = 1 & A \times 1 = A \\ A + A = A & A \times A = A \\ A + A' = 1 & A \times A' = 0 \\ A'' = A \end{array}$$

PROPRIEDADES

$$\begin{array}{ll} A + B = B + A & AB = BA \\ (A+B)+C = A+(B+C) & (AB)C = A(BC) \\ A(B + C) = AB + AC \end{array}$$

REGRAS EXCLUSIVAS

$$\begin{array}{l} A + AB = A \\ A + A'B = A + B \\ (A + B)(A + C) = A + BC \end{array}$$

DEMORGAN

$$\begin{array}{l} (AB)' = A' + B' \\ (A+B)' = A'B' \end{array}$$

XOR

$$A (+) B = A'B + AB'$$

19. Exercícios dos slides, resolvidos

E1. Escreva a expressão do circuito e simplifique — $AB + BC(B + C)$

$$Q = AB + BC(B + C)$$

$$\begin{aligned}
&= AB + BCB + BCC && \text{distributiva} \\
&= AB + BC + BC && \text{pois } BB = B \text{ e } CC = C \text{ (regra } A \times A = A) \\
&= AB + BC && \text{pois } A + A = A \\
&= B(A + C) && \text{fatorando } B
\end{aligned}$$

Resposta: $Q = B(A + C)$. De **cinco portas para duas**. Para conferir, monte a tabela-verdade das duas expressões nas 8 combinações de A, B, C — devem ser idênticas.

E2. Desenhe o circuito de $B(A + C)$

Avalie na ordem matemática correta (parênteses primeiro, depois multiplicação):

1. $A + C \rightarrow$ uma porta **OR** de 2 entradas.
2. $B \cdot (\text{saída do OR}) \rightarrow$ uma porta **AND** de 2 entradas.

Total: 2 portas, contra 5 do circuito original.

E3. Reduza $(A + (BC)')'$ por DeMorgan

Resolvido na seção 8.3. **Resposta:** $A'BC$ — AND de 3 entradas com A invertido.

Caminho alternativo (quebrando a barra curta primeiro):

$$\begin{aligned}
&(A + (BC)')' \\
&= (A + B' + C')' && \text{quebrou a barra de BC} \\
&= A' \cdot B'' \cdot C'' && \text{quebrou a barra longa (em 3 lugares - permitido)} \\
&= A' \cdot B \cdot C = A'BC
\end{aligned}$$

Mesmo resultado. Note que no segundo passo a barra longa foi quebrada em **três lugares ao mesmo tempo** — isso é legítimo, porque é **uma única barra**. O proibido é quebrar **duas barras diferentes** no mesmo passo.

E4. SOP da lógica “dois de três”

Resolvido na seção 10.2. $A'BC + AB'C + ABC' + ABC \rightarrow AB + BC + AC$.

E5. POS do detector de falha de sensor

Resolvido na seção 10.3. $(A + B + C)(A' + B' + C')$.

E6. Marque a célula de AB no K-map de 2 variáveis

1. Sombreie/circule a região correspondente a A.
2. Sombreie/circule a região correspondente a B.
3. A **sobreposição** das duas é $AB \rightarrow$ coloque **1** nessa célula.

No K-map de 2 variáveis com A nas colunas e B nas linhas, é a célula **inferior direita** ($A=1, B=1$). Não é preciso deixar a marcação de sombreado.

E7. Identificar células num K-map de 3 variáveis

Com $2^3 = 8$ células, cada uma é identificada unicamente por um termo produto:

- $A'B'C' \rightarrow$ célula do **canto superior esquerdo**
- $ABC' \rightarrow$ célula **mais à direita na parte inferior**

E8. Transferir tabela-verdade para o K-map e extrair a expressão

Procedimento dos slides, passo a passo:

1. Conte os 1s na tabela — o K-map deve ter todos.
2. Localize o primeiro 1; anote o endereço **AB** daquela linha.
3. Ache a célula do K-map com o **mesmo endereço**; escreva 1.
4. Repita para os demais 1s.
5. Agrupe células adjacentes (nunca na diagonal).
6. Identifique as variáveis **iguais** dentro do grupo → elas ficam.
7. Descarte as variáveis que **variam** no grupo.
8. Ignore variáveis que não aparecem em nenhum 1.

No exemplo dos slides, dois 1s na mesma coluna: B é igual em toda a coluna (fica), A varia entre 0 e 1 (sai), B' não aparece (ignorado). **Resposta: Saída = B.**

E9. Os agrupamentos de 4 variáveis dos slides

mapa	agrupamento	resposta
6 termos p	grupo superior de 4 + 2 inferiores	$A' + B$
4 termos p	juntadas a 2 do grupo anterior	
	enrolar as extremidades como	C'
	cilindro → grupo de 4 com $C = 0$	
4 cantos	dobrar os cantos como	$B'D'$
	guardanapo	
8 células	enrolar topo/base → $B = 0$ nas	B'
	oito	
9 termos p	dois grupos de 8 compartilhando	$B' + D'$
	os cantos	
3 pares + 1 solta	a solta não combina com nada	$B'C'D' + A'B'D' + ABCD$

Sobre o penúltimo: compartilhar células entre os dois grupos de 8 **leva a uma solução melhor** do que formar um grupo de 8 e um de 4 sem compartilhar. Confirma a regra “reutilizar é desejável”.

20. Simulado (15 questões)

Responda antes de olhar o gabarito da seção 20.1.

1. Por que $1 + 1 = 1$ na álgebra booleana? a) Porque a soma satura em 1 b) Porque só existem 1 e 0, e a soma não é 0, logo é 1 por eliminação c) Porque $1 + 1 = 2$ e $2 \bmod 2 = 0$, invertido d) Porque a porta OR tem ganho unitário
2. Qual afirmação sobre booleano x binário é correta? a) São a mesma coisa com nomes diferentes b) Booleano usa vários bits ponderados por posição c) Binário é notação para números reais; booleano é um sistema matemático distinto, de 1 bit d) Binário só existe dentro da álgebra booleana
3. $(AB)'$ é igual a: a) $A'B'$ b) $A' + B'$ c) AB d) $(A + B)'$
4. Qual destas operações **não existe** na álgebra booleana? a) Adição b) Multiplicação c) Complementação d) Subtração
5. A regra $A + A'B = A + B$ é: a) A propriedade distributiva b) Uma regra exclusiva da álgebra booleana c) Um caso particular de DeMorgan d) Válida também nos números reais

6. A sequência correta no topo de um mapa de Karnaugh de 4 variáveis é: a) 00, 01, 10, 11 b) 00, 01, 11, 10 c) 11, 10, 01, 00 d) 00, 11, 01, 10
7. Num K-map de 4 variáveis, um grupo de 8 células gera um termo com: a) 4 variáveis b) 3 variáveis c) 2 variáveis d) 1 variável
8. As quatro células dos cantos de um K-map de 4 variáveis: a) nunca podem ser agrupadas b) formam um grupo válido de 4, porque o mapa enrola nos dois eixos c) formam quatro grupos de 1 d) só se agrupam se houver *don't care*
9. Ao mapear um **maxtermo**, o endereço da célula é: a) o valor binário direto do termo b) o **complemento** do valor binário do termo c) sempre a célula superior esquerda d) indiferente
10. Uma condição *don't care* deve ser incluída num grupo quando: a) sempre — X conta como 1 b) nunca — X conta como 0 c) apenas se isso permitir formar um grupo **maior** d) apenas em soluções POS
11. $A (+) B$ (XOR) equivale a: a) $AB + A'B'$ b) $A'B + AB'$ c) $(A + B)'$ d) $A' + B'$
12. Na conversão tabela-verdade $\rightarrow$ SOP, olham-se as linhas cuja saída é: a) 0 b) 1 c) X d) qualquer uma
13. Ao aplicar DeMorgan numa expressão com várias camadas de barras, o procedimento correto é: a) quebrar todas as barras de uma vez b) quebrar duas barras por passo para economizar tempo c) quebrar uma barra por vez, começando pela mais externa d) quebrar sempre a mais interna primeiro, jamais a externa
14. O circuito “dois de três” do incinerador, simplificado, é: a) ABC b) $A + B + C$ c) $AB + BC + AC$ d) $A'BC + AB'C$
15. Duas coberturas diferentes de um mesmo K-map resultaram, ambas, em quatro termos de três variáveis. Isso significa que: a) uma delas está errada b) as duas são soluções de custo mínimo igualmente válidas c) o mapa foi montado em ordem binária d) faltou agrupar os cantos

20.1 Gabarito

Q	Resp.	Por quê
1	b	Só há dois valores; a soma não é 0, logo é 1 por eliminação
2	c	Booleano = sistema matemático de 1 bit; binário = notação posicional para reais
3	b	DeMorgan. $(AB) = A' + B'$. A alternativa (a) é a pegadinha clássica
4	d	Subtração implica negativos, proibidos no booleano (divisão também não existe)
5	b	Não tem equivalente nos reais; prova: $(A+A')(A+B) = 1 \cdot (A+B)$
6	b	Código Gray — muda um bit por posição
7	d	Grupo de 8 em 4 variáveis elimina 3 $\rightarrow$ sobra 1 variável
8	b	“Dobre os cantos como um guardanapo”: grupo de 4 válido $\rightarrow$ tipicamente $B'D'$

Q	Resp.	Por quê
9	b	No POS o endereço é complementado — o passo que todo mundo esquece
10	c	O X é ativo mas estratégico : só entra se aumentar o grupo
11	b	Complementos cruzados . Substitui 2 ANDs + 1 OR por 1 XOR
12	b	SOP olha os 1s ; POS olha os 0s
13	c	Uma barra por passo, da mais externa para dentro. Quebrar uma barra em vários lugares é permitido; duas barras num passo, não
14	c	Função maioria: $A'BC + AB'C + ABC' + ABC = AB + BC + AC$
15	b	Mais de uma solução de custo mínimo é comum e prevista nos slides

Checklist de véspera

- ☐ Sei dizer **por que** $1 + 1 = 1$ sem decorar (argumento por eliminação)
- ☐ Não confundo booleano com binário
- ☐ Sei as 8 identidades de cabeça
- ☐ Sei que $A + 1 = 1$ pode ser aplicado com $A =$ um termo inteiro ($ABC + 1 = 1$)
- ☐ Escrevo DeMorgan nas duas formas sem hesitar
- ☐ Nunca quebro duas barras num passo
- ☐ Sei escrever XOR como $A'B + AB'$ e reconhecer a forma num circuito
- ☐ Sei montar SOP (linhas com 1) e POS (linhas com 0) sem trocar
- ☐ Escrevo o topo do K-map em Gray **automaticamente**
- ☐ Lembro de checar bordas e cantos antes de fechar os grupos
- ☐ Reutilizo células para formar grupos maiores
- ☐ No POS, **complemento** o endereço
- ☐ Trato X como 0 ou 1 conforme convier ao tamanho do grupo
- ☐ Aceito que pode haver mais de uma resposta mínima correta